Rendu du TP 1 DevOps (Git) - Livrables de l'équipe

TP 1 : Synchronisation, branches, fusions et résolution de conflits

Exercice 1 : Origin et Remotes

1- Affichez les remotes avec leurs URL.

- $ git remote -v
- origin https://github.com/LabrouziFatima-Ezzahrae/ArtiShop (fetch)
- origin https://github.com/LabrouziFatima-Ezzahrae/ArtiShop (push)

  le remote origin pointe vers https://github.com/LabrouziFatima-Ezzahrae/ArtiShop.git

3- Combien de branches distantes voyez-vous, et pour quels remotes ?

on voit 6 branches distantes au total réparties sur deux remotes :

Pour le remote origin : origin/main, origin/feature/livraison, et origin/feature/promo.
Pour le remote upstream : upstream/main, upstream/feature/livraison, et upstream/feature/promo.

5. Tentez git push upstream main. Que se passe-t-il ?

⇒ La commande est rejetée avec une erreur de permission : Permission denied et error: 403.

Q1.1 Dans quel fichier les remotes sont-ils enregistrés ?

Les remotes sont enregistrées dans le fichier caché nommé .git/config, situé à la racine de dossier de projet local.

Q1.2 Pourquoi le push vers upstream est-il refusé ?

Le push vers upstream est refusé car ce dépôt est en lecture seule ; seul le propriétaire a les droits d'écriture.

Exercice 2 : les clones de l'équipe

Q2.1 Quel remote existe déjà dans le clone, et pourquoi ? upstream y figure-t-il ?

Le remote origin existe déjà dans le clone. git clone le crée automatiquement et le fait pointer vers l'URL du dépôt cloné, git push et git pull fonctionnent sans préciser de destination : ils utilisent origin par défaut.

upstream n'y figure pas. Ce remote n'est pas créé par le clone. Il faut l'ajouter manuellement avec git remote add upstream <URL>. Dans ce clone, git remote -v n'affiche donc que :

origin https://github.com/<compte-membre1>/ArtiShop.git (fetch)

origin https://github.com/<compte-membre1>/ArtiShop.git (push)

Exercice 3 : fetch ou pull ?

git fetch origin, puis git status, git log --oneline

main..origin/main et grep Recette README.md.

2- Qu'observez-vous à chaque commande

git fetch origin

Git a téléchargé le nouveau commit depuis le fork. La ligne main -> origin/main montre que seule la référence origin/main a avancé.

git status

Le message Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded signifie que votre main a 1 commit de retard sur origin/main. Il n'y a pas de divergence, donc une fusion fast-forward serait possible. working tree clean confirme que vos fichiers n'ont pas été modifiés.

git log --oneline main..origin/main

Cette commande liste les commits présents sur origin/main mais absents de main. Il y en a un seul : ajout une section Recette QA, le commit de Membre 4.

grep Recette README.md

cette commande ne devrait rien afficher : le fetch n'a pas modifié README local, la section de Membre 4 n'est donc pas encore dans le fichier.

grep affiche pourtant Recette QA. Ce n'est pas cohérent avec le git status propre. Ce mot existait peut-être déjà dans README avant le fetch,

Q3.1 : À quelles deux commandes équivaut `git pull` ?

`git pull` équivaut à l'enchaînement de deux commandes distinctes :

1. **`git fetch`** : pour télécharger et récupérer les dernières modifications du dépôt distant sur machine (dans les références distantes).
2. **`git merge`** : pour fusionner automatiquement ces modifications distantes dans votre branche locale courante.

Q3.2 : Pourquoi `git fetch` est-il qualifié de commande « sûre » ?

`git fetch` est qualifiée de « sûre » parce qu'elle se contente de mettre à jour vos branches de suivi distant (comme `origin/main`) en arrière-plan, sans modifier votre code source actuel, sans toucher à l'espace de travail (*working directory*) et sans fusionner quoi que ce soit dans vos branches de travail locales. Elle ne présente donc aucun risque de modifier ou de casser le travail en cours ou d'introduire des conflits de manière inopinée.

Exercice 4 : récupérer les branches existantes

1- git fetch, git branch puis git branch -a. Pourquoi les branches qui apparaissent sont differentes ?

git branch (Branches locales uniquement) : N'affiche que les branches qui ont été créées ou extraites physiquement sur ta machine locale pour y travailler. Lors d'un clone, Git ne crée localement que la branche par défaut (ici, main).

git branch -a (Toutes les branches : locales + distantes) : L'option -a (pour all) combine les deux. Elle affiche ta branche locale (main) et les pointeurs vers les branches distantes (remotes/origin/feature/livraison, etc.).

2- Membre 4 essaie dans l'ordre les trois commandes suivantes et note à chaque fois le message et l'état de HEAD :

a) git switch origin/feature/livraison ;

La commande échoue avec une erreur (fatal: a branch is expected). git switch refuse de se placer directement sur une branche distante de suivi (qui est en lecture seule).

b) git checkout origin/feature/livraison puis git status ;

La commande réussit, mais te place dans un état critique appelé « detached HEAD » (HEAD détachée). HEAD ne pointe plus vers le nom d'une branche locale, mais pointe directement vers l'identifiant du commit (5333466).

c) git switch feature/livraison.

C'est la bonne méthode. Git crée automatiquement une branche locale feature/livraison qui va suivre les modifications de la branche distante origin/feature/livraison, et te place dessus.

3. Membre 4 revient sur main.

Pour revenir sur main : Il te suffit d'exécuter la commande git switch main.

Quel risque court-on en commitant dans l'état « detached HEAD » ?

Le risque majeur est la perte de tes commits. Si tu effectues des modifications et que tu les commites en état de HEAD détachée, ces nouveaux commits ne seront attachés à aucune branche. Dès que tu changeras de branche (par exemple, en faisant un git switch main), ces commits deviendront "orphelins" (invisibles dans l'historique normal).

Exercice 5 : codes promo et fusion fast-forward (Membre 2)

Q5.1 Pourquoi Git n'a-t-il pas créé de commit de fusion, ni à l'étape 2 ni à l'étape 6 ?

Git n’a pas créé de commit de fusion car, dans les deux cas, la fusion était de type fast-forward : les deux branches n’avaient pas divergé. La branche à fusionner était simplement en avance sur la branche courante, sur la même ligne d’historique. Git a donc seulement déplacé le pointeur de la branche courante vers le dernier commit, sans créer de nouveau commit.

Étape 2 (git merge main sur feature/promo) : feature/promo n’avait aucun commit propre, et main avait un commit de plus (la section « Recette QA »). Git a avancé feature/promo.

Étape 6 (git merge feature/promo sur main) : main n’avait reçu aucun nouveau commit entre-temps, et feature/promo contenait un commit de plus. Git a avancé main de 6602890 à 1879e58.

Exercice 6 : frais de livraison et fusion à 3 voies (Membre 3)

Q6.1 Quels sont les trois commits utilisés par cette fusion ? Utilisez git merge-base main feature/livraison pour le vérifier.

Une fusion à 3 voies (3-way merge) utilise trois commits pour générer le commit de fusion :

Le commit de base commun (merge base), qui est le dernier ancêtre commun aux deux branches (trouvé via la commande git merge-base main feature/livraison).

Le tip de la branche actuelle (HEAD / main).

Le tip de la branche à fusionner (feature/livraison).

Q6.2 Membre 2 et Membre 3 ont tous deux modifié src/app.js. Pourquoi Git a-t-il pu fusionner sans conflit ?

Git a pu fusionner sans conflit car le Membre 2 et le membre 6 avez modifié des parties différentes du fichier src/app.js (le Membre 2 a ajouté les codes promo dans les routes du panier/gestion des codes, tandis que tu as ajouté la route /api/delivery près de /health). Git est assez intelligent pour combiner automatiquement des modifications apportées à des lignes différentes d'un même fichier.

Exercice 7 : conflit sur RAMADAN20

Q7.1 Pourquoi une fusion fast-forward ne produit-elle jamais de conflit ?

Une fusion `fast-forward` (avance rapide) ne produit jamais de conflit pour les raisons suivantes :

Absence de divergence : Pour qu'une fusion soit de type fast-forward, la branche cible (par exemple main) n'a reçu aucun nouveau commit depuis le moment où la branche de travail a été créée à partir d'elle. L'historique reste strictement linéaire.

Pas de modifications concurrentes : Git n'a pas besoin de arbitrer ou de combiner deux états du code qui ont évolué en parallèle. Il se contente simplement de déplacer le pointeur de la branche cible (HEAD) pour qu'il pointe directement vers le commit le plus récent de la branche fusionnée.

Par conséquent, comme il n'y a qu'une seule lignée de modifications, aucun conflit de contenu ne peut techniquement survenir.

Exercice 8 : supprimer des branches

3. Ces branches existent-elles encore chez upstream ? Vérifiez avec `git ls-remote --heads upstream`.

Oui, ces branches existent toujours chez upstream. Explication : Comme le prouve le résultat de la commande git ls-remote --heads upstream, les branches feature/livraison et feature/promo sont toujours présentes sur le dépôt source. Les suppressions effectuées à l'étape 1 ont uniquement affecté les dépôts locaux des membres et le fork de l'équipe (origin). Notre équipe n'a qu'un accès en lecture seule sur le dépôt de l'enseignante (upstream), nous ne pouvons donc pas y supprimer de branches.

Exercice 9 : `git pull --rebase` (Membre 2 et Membre 4)

Q9.1 Qu'aurait donné `git pull --no-rebase` ?

- Git aurait fait un merge : un commit de fusion aurait été créé, avec deux parents (7822e54 et af07d24). Le graphe aurait montré un losange, et ton commit aurait gardé son hash d'origine 7822e54.

Q9.2 Quelle est la « règle d'or » du rebase ?

- Ne jamais rebaser des commits déjà poussés et partagés avec d'autres personnes. Le rebase réécrit l'historique (nouveaux hash), ce qui désynchronise les autres membres et oblige à forcer les pushs. On ne rebase que ses commits locaux, pas encore publiés. Ici, ça marche car ton commit 7822e54 n'avait jamais été poussé.

Exercice 10 : rester synchronisé avec upstream (Membre 1)

1. Membre 1 lance git fetch upstream, puis git log --oneline main..upstream/main et git log - oneline upstream/main..main. Que représente chacune de ces listes ?

`git log --oneline main..upstream/main` (vide) : Le fait qu'il n'y ait aucun résultat signifie que l'enseignante n'a ajouté aucun nouveau code sur son dépôt d'origine (upstream/main). le dépôt local est parfaitement à jour par rapport au sien. git log --oneline upstream/main..main : Cette liste représente tout le travail de l équipe qui se trouve sur mon ordinateur , mais qui n'est pas chez l'enseignante (upstream).

On y voit tout l'historique de TP : la section Recette QA, les codes promos, les frais de livraison, les fusions, la résolution du conflit sur le taux de 25% vs 20%, et même le récent changement de couleur (style)

3. Sur GitHub, repérez le bouton Sync fork de votre fork. Que fait-il en ligne de commande ?

Sur l'interface GitHub, ce bouton effectue automatiquement l'équivalent d'un git fetch upstream suivi d'un git merge upstream/main directement sur mon dépôt distant, me permettant d'être à jour sans passer par le terminal.

Exercice 11 (bonus) : tags et push forcé

1. Membre 1 crée le tag annoté v1.1.0 (codes promo et livraison), fait `git push`, puis regarde `git ls remote --tags origin`. Que constate-t-il ?

Ce que l'on constate : La commande git ls-remote --tags origin affiche deux lignes pour le même tag v1.1.0. C'est parce que nous avons créé un tag "annoté" (avec l'option -a).

● La première ligne (refs/tags/v1.1.0) correspond à l'identifiant (hash) de l'objet "tag" lui-même, qui contient le message "Codes promo et livraison".

● La deuxième ligne avec le symbole ^{} correspond à l'identifiant du commit réel sur lequel le tag a été posé (le commit sous-jacent).

Q11.1 Pourquoi utiliser --force-with-lease plutôt que --force ? Sur quelles branches ne force-t-on jamais ?

`--force-with-lease` est plus sûr que --force car il vérifie d'abord que personne d'autre n'a poussé de code sur la branche distante entre-temps. S'il y a du code inconnu en ligne, il refusera d'écraser le travail de tes collègues. On ne force jamais sur les branches partagées et protégées, comme la branche main

Graphe de l'historique Git du projet (git log --oneline --graph --all --decorate)

```
PS C:\Users\Fatima-ezzahrae\IdeaProjects\ArtiShop> git log --oneline --graph --all --decorate
* f2e1420 (origin/feature/avis) ajout des avis clients
* a6ffb92 (HEAD -> main, tag: v1.1.0, origin/main, origin/HEAD) style: nouvelle couleur principale
* af07d24 fix une phrase dans README
* 0ba9909 Test conflit : passage à 40% sur main
* 60a09e0 Merge branch 'fix/taux-ramadan' : résolution du conflit et correction de syntaxe
*   77516d1 Merge branch 'fix/taux-ramadan' : résolution du conflit sur RAMADAN20
|\
| * f01fce2 fix(promo): mise à jour du taux RAMADAN20 à 25%
* |   d48bef4 Merge branch 'feature/promo'
|\ \
| |/
|/|
| * 4fdee36 fix(promo): préciser le taux RAMADAN20 (validé par la PO)
* |   8f91487 Merge branch 'feature/livraison'
|\ \
| |/
|/|
| * a62d5b2 feat(livraison): frais de livraison par ville
| * 5333466 (upstream/feature/livraison, origin/feature/livraison) feat(livraison): frais de livraison par ville
* | 1879e58 feat(promo): codes promo BIENVENUE10 et RAMADAN20
* | 6602890 ajout une section Recette QA
* | c923300 (upstream/main, upstream/HEAD) ReadMe Update
|/
*   c0e265d (upstream/feature/promo, origin/feature/promo) Fusion avec le dépôt GitHub initial
|\
| * 865adb5 Initial commit
* add0021 premier commit Artishop
```